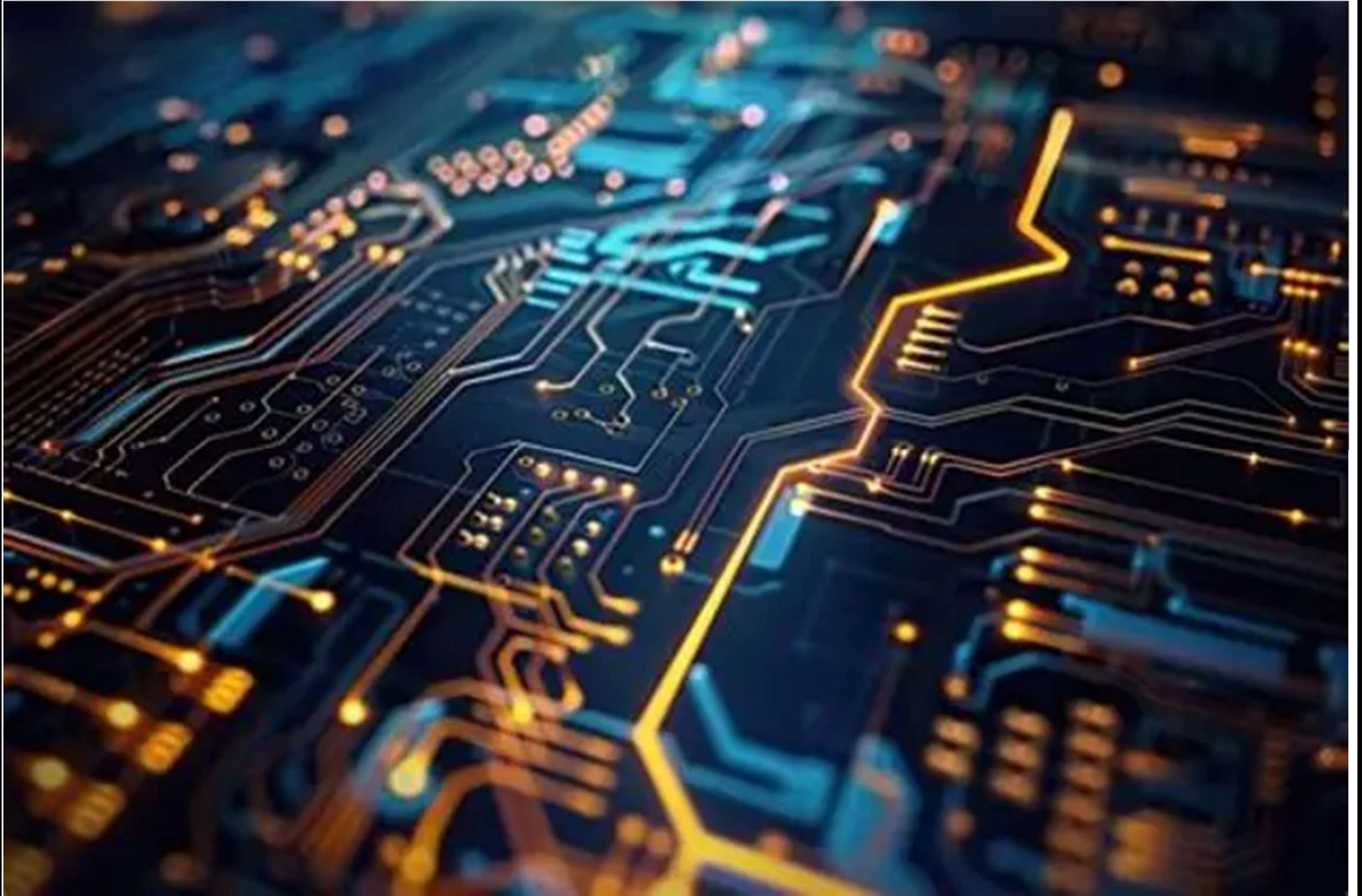


lab: FSM debouncer



Name: Ahmed Wael Sadek El
Sawy

lab: FSM debouncer

FSM debouncer

fsm_debouncer.v

```
module debouncer_fsm#(
    parameter TICK_CYCLES = 5
)((
    input  wire clk,
    input  wire rst_n,
    input  wire sw,
    output reg  db
);
    reg [$clog2(TICK_CYCLES)-1:0] counter;
    reg m_tick;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            counter <= 0;
            m_tick <= 1'b0;
        end else begin
            if (counter == TICK_CYCLES - 1) begin
                counter <= 0;
                m_tick <= 1'b1;
            end else begin
                counter <= counter + 1;
                m_tick <= 1'b0;
            end
        end
    end

    localparam S_ZERO      = 3'b000;
    localparam S_WAIT1_1   = 3'b001;
    localparam S_WAIT1_2   = 3'b010;
    localparam S_WAIT1_3   = 3'b011;
    localparam S_ONE       = 3'b100;
    localparam S_WAIT0_1   = 3'b101;
    localparam S_WAIT0_2   = 3'b110;
    localparam S_WAIT0_3   = 3'b111;

    reg [2:0] cs, ns;

    // Next State Logic (Combinational)
    always @(*) begin
        case(cs)
```

lab: FSM debouncer

```
S_ZERO:
    if (sw) ns = S_WAIT1_1;
    else    ns = S_ZERO;

S_WAIT1_1:
    if (!sw)      ns = S_ZERO;
    else if(m_tick) ns = S_WAIT1_2;
    else          ns = S_WAIT1_1;

S_WAIT1_2:
    if (!sw)      ns = S_ZERO;
    else if(m_tick) ns = S_WAIT1_3;
    else          ns = S_WAIT1_2;

S_WAIT1_3:
    if (!sw)      ns = S_ZERO;
    else if(m_tick) ns = S_ONE;
    else          ns = S_WAIT1_3;

S_ONE:
    if (!sw) ns = S_WAIT0_1;
    else    ns = S_ONE;

S_WAIT0_1:
    if (sw)      ns = S_ONE;
    else if(m_tick) ns = S_WAIT0_2;
    else          ns = S_WAIT0_1;

S_WAIT0_2:
    if (sw)      ns = S_ONE;
    else if(m_tick) ns = S_WAIT0_3;
    else          ns = S_WAIT0_2;

S_WAIT0_3:
    if (sw)      ns = S_ONE;
    else if(m_tick) ns = S_ZERO;
    else          ns = S_WAIT0_3;

    default: ns = S_ZERO;
endcase
end

// State Memory (Sequential)
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) cs <= S_ZERO;
```

lab: FSM debouncer

```
        else          cs <= ns;
    end

    // Output Logic (Combinational - Moore FSM)
    always @(*) begin
        if (cs == S_ONE || cs == S_WAIT0_1 || cs == S_WAIT0_2 || cs == S_WAIT0_3)
            db = 1'b1;
        else
            db = 1'b0;
        end
    end
endmodule
```

tb_debouncer.v

```
module tb_debouncer;

    // 50MHz clock -> 20ns period
    reg clk;
    reg rst_n;
    reg sw;
    wire db;

    integer errors = 0;

    debouncer_fsm #(
        .TICK_CYCLES(5)
    ) DUT (
        .clk(clk),
        .rst_n(rst_n),
        .sw(sw),
        .db(db)
    );

    // Clock generation (20ns period)
    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end

    // Stimulus and Self-Checking Logic
    initial begin
        rst_n = 0;
        sw = 0;
        @(negedge clk);
        rst_n = 1;
    end
endmodule
```

lab: FSM debouncer

```
repeat(5) @(negedge clk);

//Switch Bounce / Glitch Rejection (0 to 1)

sw = 1;
// Hold for 12 cycles (240ns).
repeat(12) @(negedge clk);
sw = 0;

repeat(5) @(negedge clk);

if (db !== 1'b0) begin
    $display("ERROR [Scen 1]: FSM failed to reject a high glitch. db
should be 0, got %b", db);
    errors = errors + 1;
end else begin
    $display("PASS [Scen 1]: High glitch successfully rejected.");
end

//Stable Press (0 to 1)

sw = 1;
// Hold for 20 cycles (400ns).
repeat(20) @(negedge clk);

if (db !== 1'b1) begin
    $display("ERROR [Scen 2]: FSM failed to register a stable press. db
should be 1, got %b", db);
    errors = errors + 1;
end else begin
    $display("PASS [Scen 2]: Stable press successfully registered.");
end

//Switch Bounce / Glitch Rejection (1 to 0)
sw = 0;
// Hold for 12 cycles (240ns).
repeat(12) @(negedge clk);
sw = 1;

repeat(5) @(negedge clk);

if (db !== 1'b1) begin
```

lab: FSM debouncer

```
$display("ERROR [Scen 3]: FSM failed to reject a low glitch. db
should remain 1, got %b", db);
    errors = errors + 1;
end else begin
    $display("PASS [Scen 3]: Low glitch successfully rejected.");
end

// Stable Release (1 to 0)

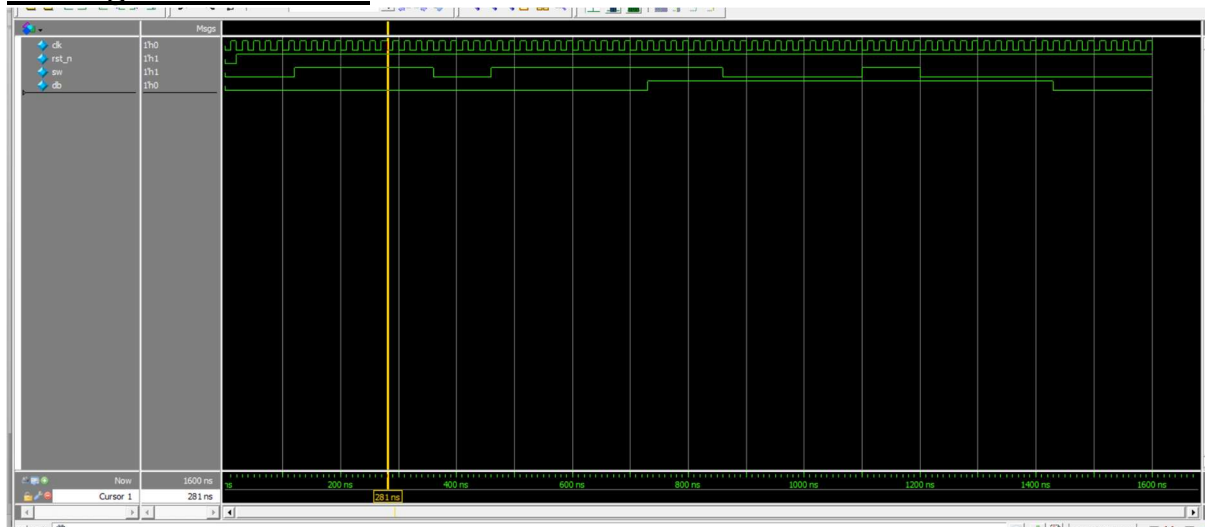
sw = 0;

repeat(20) @(negedge clk);

if (db !== 1'b0) begin
    $display("ERROR [Scen 4]: FSM failed to register a stable release. db
should be 0, got %b", db);
    errors = errors + 1;
end else begin
    $display("PASS [Scen 4]: Stable release successfully registered.");
end

if (errors == 0)
    $display("SUCCESS: All debouncer scenarios passed.");
else
    $display("FAILED: System finished with %0d errors.", errors);
$stop;
end
endmodule
```

Testing fsm debouncer



lab: FSM debouncer

```
# PASS [Scen 1]: High glitch successfully rejected.  
# PASS [Scen 2]: Stable press successfully registered.  
# PASS [Scen 3]: Low glitch successfully rejected.  
# PASS [Scen 4]: Stable release successfully registered.  
# SUCCESS: All debouncer scenarios passed.  
# ** Note: $stop      : tb_debouncer.v(99)  
#      Time: 1600 ns  Iteration: 1  Instance: /tb_debouncer
```
